

TECHNICAL EVALUATION REPORT: EML-KAN EDGE INFERENCE

Date: July 4, 2026
Subject: Standalone Microcontroller Classification via Hybrid CNN & Kolmogorov-Arnold Networks

1. Executive Summary

This report evaluates the execution metrics, parameter footprint, and numerical stability of the **EML-KAN + MobileNetV3-Small** hybrid classifier model designed for ESP32 deployment on CIFAR-100. By replacing standard Multi-Layer Perceptron (MLP) layers with Kolmogorov-Arnold Network (KAN) layers utilizing Exponential-Logarithmic (EML) activations, we have established a highly efficient, mathematically dense classification boundary directly on edge hardware.

2. Hard Parameters & Compression Statistics

The primary goal of this architecture was to minimize the parameters of the classification head while retaining high representational power. Below is the direct parameter comparison against the official PyTorch MobileNetV3-Small config:

Model Segment	Standard MLP Head	EML-KAN Head (K=2)	EML-KAN (70% Pruned DAG)
Layer Structure	576 -> 1024 -> 100	576 -> 100	576 -> 100 (Pruned)
Active Params	693,348	58,600	17,580
Flash Memory Size	2.77 MB	234.4 KB	70.3 KB
Compression Ratio	Baseline (1x)	11.8x compression	39.4x compression

3. Real-Time Execution Statistics (ESP32 vs Host PC)

- Classification Latency (EML-KAN Head):** 9.91 milliseconds (9,915 microseconds) on the ESP32.
- Input-to-Prediction Transmission:** 2,304 bytes transmitted over USB serial at 115200 baud in ~200 ms.
- Transmission Verification Integrity:** 100% alignment. The de-quantized features received on the ESP32 matched the local PyTorch features to within $< 10^{-6}$ precision, indicating zero byte drops or endian corruption.

- **Model Convergence Accuracy:** The hybrid training pipeline on the remote L40S GPU consistently converges to **97.00%** training accuracy and **79.09%** test accuracy on CIFAR-100.

4. Architectural Bottlenecks & Brutal Realities

While the EML-KAN classifier successfully maps complex decision boundaries with a 39x parameter reduction, the deployment revealed critical constraints that must be handled:

1. Numerical Instability of KANs: Standard Softplus activations $\log(1 + e^z)$ cause immediate single-precision float overflows to infinity (inf) on the ESP32 if $z > 88.72$. This saturates the model outputs and forces constant predictions (e.g., class 23 for all inputs). We resolved this by implementing a custom numerically stable function:

```
inline float softplus_val(float z) {  
    if (z > 20.0f) return z;  
    if (z < -20.0f) return 0.0f;  
    return logf(1.0f + expf(z));  
}
```

2. Computational Overhead of Non-Linear Functions: Executing transcendentals like expf and logf inside a loop 200 times per inference is computationally heavy for microcontrollers, contributing to the 9.91 ms latency.

5. Future Roadmap: Standalone Edge Execution

- **Int8 Quantization (TFLite Converter):** Compress the full MobileNetV3 backbone to 8-bit integers, reducing the total model footprint from 3.9 MB to 1.0 MB to fit within the standard 4MB ESP32 flash.
- **Local Frame Capture:** Integrate the ESP32-CAM frame buffer directly with the TFLite Micro input tensor to eliminate PC-serial dependency entirely.
- **Hardware Acceleration:** Utilize Espressif's optimized ESP-NN libraries inside TFLite Micro to accelerate Int8 convolutions, targeting a full pipeline execution latency of < 250 ms directly on the edge.